Umm AL- Qura University

AI3104 — AI System Design

AI System Design Project

Phase 1: Problem Definition & Planning

Project Name: Saudi Unemployment Rate Forecasting System

Name	Student ID	Group
Mirana Alghamdi	445005808	1
Aram Alouni	44510225	1
Khawlah Aljuaid	445001074	1
Amasi Alsahbi	445005698	1

Supervisor: Dr. Hwazen Bdwi

Table of Contents

Phase 1 : Introduction

1.1 Abstract

Saudi Arabia's labor market is one of the most closely monitored economic indicators in the Kingdom, directly tied to Vision 2030's target of reducing the national unemployment rate to 7% by 2030. Despite the availability of official quarterly data through DataSaudi, to the best of our knowledge, no publicly available Saudi platform currently provides AI-driven forecasting of future national unemployment rates through an interactive dashboard for policymakers and researchers. This project develops a complete AI-based forecasting system that ingests historical quarterly national unemployment data from Q2 2016 to Q4 2025, trains and compares three models (Linear Regression as baseline, Random Forest, and SARIMA), selects the best performer, and deploys it as an interactive Streamlit dashboard hosted on Streamlit Cloud. SARIMA achieved the best results with a MAE of 0.46 percentage points and R^2 of 0.45 on the test set. The system is designed for government planners, researchers, and analysts who need forward-looking labor market intelligence to support evidence-based decisions aligned with Vision 2030

1.2 Problem Statement and background context

- Unemployment is an important issue in Saudi Arabia because it affects both the economy and society.
- Most available unemployment data focuses on historical trends and does not provide future forecasts.
- There is a need for an AI-based system that can analyze historical labor market data and predict future unemployment rates.
- This project uses historical unemployment data from DataSaudi to forecast national unemployment rates for upcoming quarters.
- The system presents predictions through a simple and interactive dashboard to support decision-making.
- The proposed solution aligns with Saudi Vision 2030 goals related to reducing unemployment and supporting labor market development.

1.3 Target Users and Success Metrics (KPIs)

Target Users

This system primarily targets government entities and decision-makers in Saudi Arabia, especially those responsible for economic planning and labor market policies. These users require accurate forecasts to support data-driven decision-making aligned with Vision 2030, particularly in reducing unemployment rates. Researchers and analysts can also benefit by studying national unemployment trends. University students may use the platform for academic and research purposes.

Success Metrics (KPIs)

- Prediction Accuracy: MAE below (1.0) percentage point on unseen test data
- Model Generalization: Positive R^2 on the test set (model outperforms a naive mean predictor)
- Data Quality: Zero missing values in the input series
- Dashboard Usability: Forecasts displayed with confidence intervals and clear disclaimers

- Response Time: Forecast generated in under (2) seconds
- Decision Support: Forecasts benchmarked against Vision 2030 target (7%)
- System Availability: Dashboard uptime should remain above 99% during operation.

1.4 Feasibility and Initial Risk Assessment

1.4.1 Feasibility Studies

Technical Feasibility: The project is technically viable. The team will use Python with statsmodels (SARIMA), scikit-learn (Random Forest, Linear Regression), pandas, and Streamlit. All libraries are open-sourced and well-documented.

Data Feasibility: The project relies on DataSaudi [1], which provides structured quarterly data with no missing values. The national Total series (39 records) is sufficient for SARIMA training and produces reliable forecasts.

Ethical Feasibility: The system uses aggregated, non-identifiable public data. All outputs are labeled as statistical projections, not guaranteed outcomes, in alignment with SDAIA AI [3] Ethics Principles.

1.4.2 Initial Risk Assessment

Risk Category	Potential Risk	Mitigation Strategy
Technical	Poor model performance or low prediction accuracy	Compare multiple models; select the best based on MAE and R^2
Data	insufficient records for complex ML models	Try another model which is purpose-built for short time series
Ethical	Misinterpretation of forecasts as guarantees	Include disclaimer on forecast outputs; label as statistical projections
Deployment	Dashboard service interruption or deployment failure	Maintain backup deployment files and redeploy the application when necessary

1.5 Ethical, Legal, Societal Considerations, assumptions, and constraints

We made sure that our project aligns with SDAIA's AI Ethics Principles [3], particularly the principles of transparency, fairness, and accountability. To align with SDAIA's principles, we want to make all model outputs clearly labeled as statistical forecasts, and prediction logs will be maintained for monitoring purposes.

Data Protection & Privacy

All data we will use in this project is sourced from DataSaudi, it is an open and official Saudi data government data platform, after reviewing the data available on DataSaudi, we

confirmed that all unemployment data we need are published as aggregated statistics and contains no personally identifiable information.
Our system complies with Saudi data governance standards as the source data is already publicly released by authorized government entities.

Fairness & Bias

We considered how fairness and bias could affect our results. Unemployment data reflects real societal inequalities shaped over time. Our system does not introduce new bias; it learns from historical macro patterns in the data. To ensure objectivity and prevent demographic misrepresentation, we will ensure balanced, unified visibility to the aggregate national metric without fragmentation, presenting the macro-economic reality of the Kingdom transparently.

Societal Considerations

A group of people who cannot find work or contribute to raising our country's economy is behind every percentage point. We don't see unemployment just as a number. We want our system to help government planners see a spike in youth unemployment coming before it happens, so they will have time to find solution, like expanding training programs, and to ensure the system is used responsibly, we will include disclaimer clarifying that results are statistical projections, not guaranteed outcomes.

Legal Considerations

DataSaudi data is publicly available under the Saudi Open Government Data License, which permits use for research and educational purposes including AI model training. No copyright or data licensing issues are anticipated.

Assumptions & Constraints

Our system is constrained by the quarterly update frequency of DataSaudi. We won't produce monthly or weekly forecasts.
And we assume DataSaudi data is representative and accurate. because any data quality issues at the source would propagate into the model outputs. Also, our model will be trained on past unemployment data, which means it works better when conditions are relatively stable, not in a crisis. If something unexpected happens like a global crisis, we assume the model would become unreliable and reduce forecast accuracy. because the model has never seen data from such situations before.
Also, since our training data covers Saudi Arabia only, the model is not designed to generalize to other labor markets

User-Centric AI:

This project reflects the user-centric AI concept [4] by focusing on the needs of our target users: decision-makers and policymakers. The system is designed not only to generate sequential forecasts but also to present the results in a clear and simple way that users can easily interpret. Instead of raw numbers, the dashboard uses interactive visual charts and allows users to evaluate historical trajectories alongside automated future confidence intervals, which helps them gain comprehensive macro insights and supports strategic decision-making.

Phase 2 : AI LifeCycle

2.1 Introduction & Problem Definition

The project uses AI techniques to analyze historical labor market data and predict unemployment rates in Saudi Arabia for upcoming periods.

Official platforms provide previous unemployment statistics, but the available information mainly focuses on past trends. Therefore, this project aims to predict future national unemployment rates by analyzing historical data using AI.

The system analyzes historical unemployment data and displays the results through charts and an easy-to-use dashboard to help decision-makers better understand labor market trends and support better decisions.

2.2 Data Collection and Preparation

2.2.1 Data Sources and Justification

The primary data source for model training and evaluation the model for this project is DataSaudi, the official open data portal managed by the Ministry of Economy and Planning of the Kingdom of Saudi Arabia [1].

This structure aligns directly with the project's analytical requirements, For the forecasting model, the national gender file is used, filtered to the Total category, producing a clean continuous quarterly time series of 39 observations spanning Q2 2016 to Q4 2025, providing approximately 10 years of historical observations.

We selected DataSaudi as our primary source for model training and evaluation because it is:

- **Authoritative**: Published by an official government body, ensuring reliability and accuracy.
- **Publicly licensed**: Available under the Saudi Open Government Data License, which permits use for research and educational purposes including AI model training.
- **Aggregated and non-identifiable**: Contains no personal information, raising no privacy concerns.

2.2.2 Data Collection Plan

Data collection involves downloading 1 file from DataSaudi covering the unemployment rate by gender.

2.2.3 Data Cleaning and Preprocessing Plan

The cleaning and preparation pipeline is structured as a sequence of well-defined stages:

-Schema Validation.

-Missing Value Handling.

-Outlier Detection.

-Normalization.

Feature Engineering:

Five derived features will be created to improve model inputs:

1- Lag_1: the unemployment rate from the previous quarter, to capture short-term trends.

2-Lag_4: the rate from the same quarter one year earlier, to capture seasonal patterns.

3-Rolling_Mean_4: a four-quarter moving average, smoothing noise and revealing trends.

4-YoY_Change: the year-over-year percentage point change, highlighting annual shifts.

5- A target variable created by shifting the unemployment rate one quarter forward, representing the value the model must predict.

-Data Standardization.

-Splitting.

The results of each preprocessing step, including the number of records affected and the final dataset size, are presented in phase 3.

2.2.4 Data Governance

All data used is classified as public, aggregated, and non-personal, consistent with the Saudi Open Government Data License and SDAIA AI Ethics Principles [3].

Dataset versions are tracked alongside code versions in GitHub and Google Drive, enabling reproducibility of any historical model to run.

Access to processed datasets is restricted to the project team.

2.2.5 Initial System Architecture Diagram

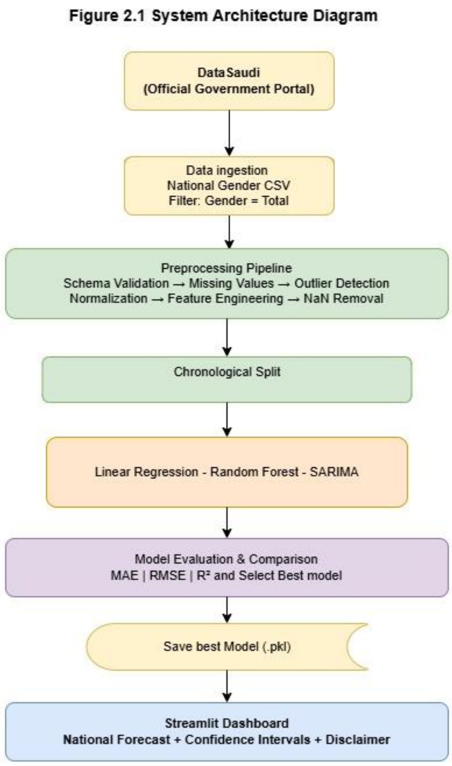


Figure 2.1

Figure 2.1 shows the Initial System Architecture Diagram. The flow starts at DataSaudi, where the CSV file is downloaded.

The data ingestion component loads and validates these files, followed by the preprocessing component that cleans, scales, and encodes the data.

Feature engineering then creates the lag, rolling mean,target, and year-over-year features. The data is split chronologically using two different strategies depending on the model type. For the supervised models (Linear Regression and Random Forest), the 34 records obtained after feature engineering are split into training (70%), validation (15%), and test (15%) sets, yielding 23, 5, and 6 records respectively. For SARIMA, which operates directly on the original time series without feature engineering, the full 39-quarter series is split from 70/30 into a training set of 27 quarters and a test set of 12 quarters. After evaluation, the best model is selected and saved as .pkl to Google Drive. The Streamlit dashboard loads this model and presents forecasts with confidence intervals to the user.

2.2.6 Experimentation and Iterative Improvement Plan

The experimentation follows four planned stages:

Stage 1: Train a Linear Regression model as the baseline to establish a minimum acceptable performance.

Stage 2: Train a Random Forest model to explore whether non-linear patterns improve accuracy.

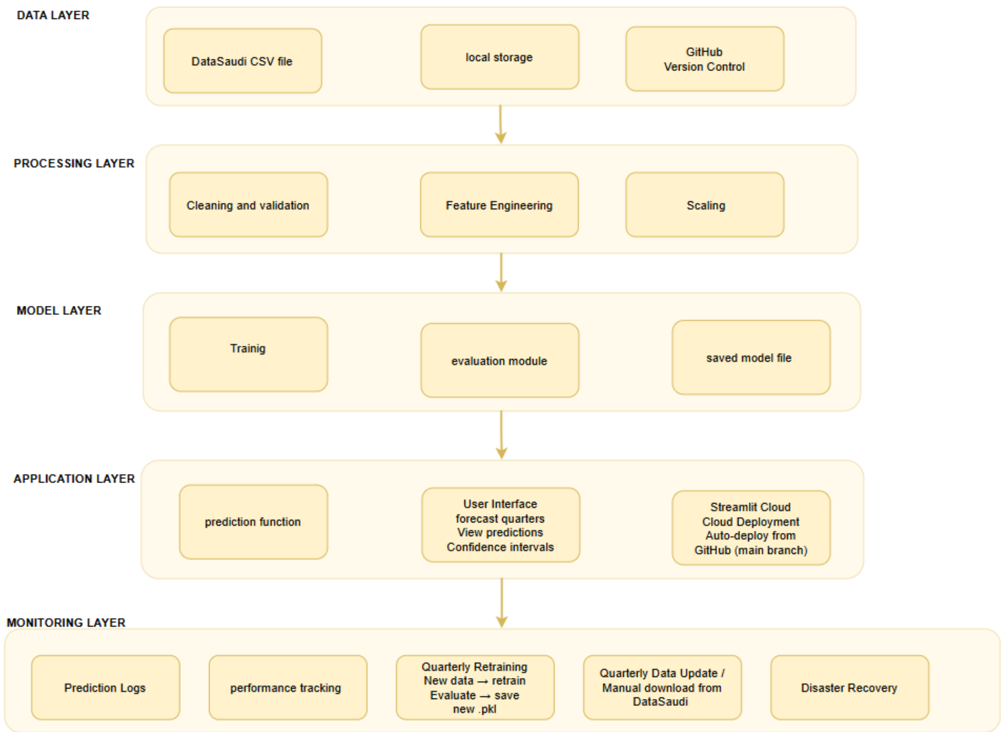
Stage 3: Apply hyperparameter tuning (GridSearchCV) to optimize the Random Forest configuration.

Stage 4: Train a SARIMA model as a statistical alternative designed for seasonal time-series data.

At each stage, the model will be evaluated using MAE, RMSE, and R^2 on the test set. If a model fails to outperform the baseline, the next stage will explore a different approach. The best performing model will be selected for deployment.

2.3 Detailed System Design

2.3.1 Logical Architecture and Components



The system will be organized into five logical layers. Each layer will have a specific responsibility and will interact with the layers above and below it.

Data Layer: This layer holds the raw input data. It includes the national gender CSV file downloaded from DataSaudi, filtered to the Total category (39 quarterly records). It also encompasses Google Drive for local storage and the GitHub repository for version control.

Processing Layer: This layer prepares the data for modeling. It handles schema validation, missing value checks, outlier detection, feature engineering (Lag_1, Lag_4, Rolling_Mean_4, YoY_Change, Target), NaN removal, and Robust Scaling. The output is clean, structured data ready for training.

Model Layer: This layer is responsible for training and evaluating the models. It includes the training pipeline, the evaluation module (MAE, RMSE, R^2), and the saved model file (sarima_model.pkl using joblib). Training is performed in Google Colab. The saved model is stored in Google Drive and acts as the bridge between offline training and online prediction.

Application Layer: This layer is the interface through which the user interacts. The prediction function loads sarima_model.pkl and generates forecasts for the next two quarters. The Streamlit dashboard displays results through five sections: forecast metrics, historical trend chart, Vision 2030 comparison, model accuracy chart, and forecast details. The dashboard is deployed on Streamlit Cloud via GitHub auto-deploy.

User Interface (Application Layer): "The dashboard is implemented as a single-page scrollable web application. Figures 2.2 through 2.6 show the five main sections of the deployed interface:

Figure 2.2

Figure 2.2 shows the Streamlit Dashboard: Main Forecast Section. The top section of the dashboard displays the system title and the two quarterly forecasts for 2026. Each forecast card shows the predicted unemployment rate, the percentage-point change relative to the previous quarter, and the confidence interval computed as ±MAE (0.4551 percentage points) around the point prediction.

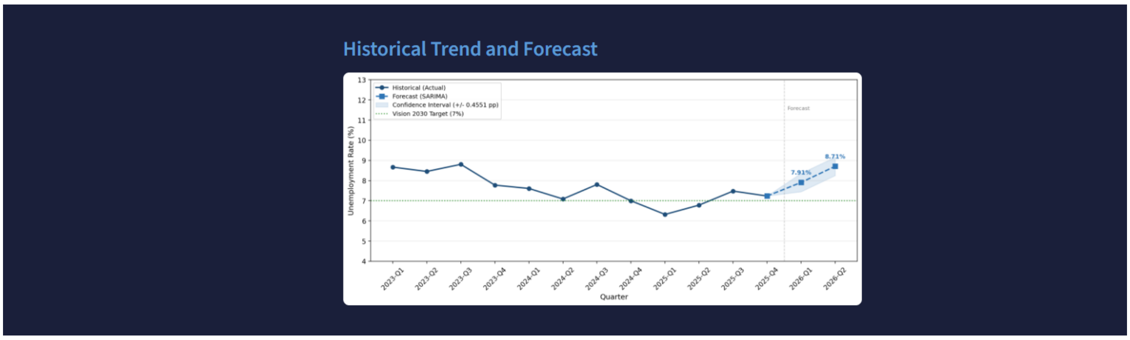


Figure 2.3

Figure 2.3 shows the Historical Trend and Forecast Chart. This chart displays the national unemployment rate from Q1 2023 to Q4 2025 (solid line) alongside the best model forecasts for Q1 2026 and Q2 2026 (dashed line). The shaded area represents the confidence interval (±0.4551 percentage points). The green dotted line marks the Vision 2030 target of 7%. The vertical dashed line separates historical data from forecast values.

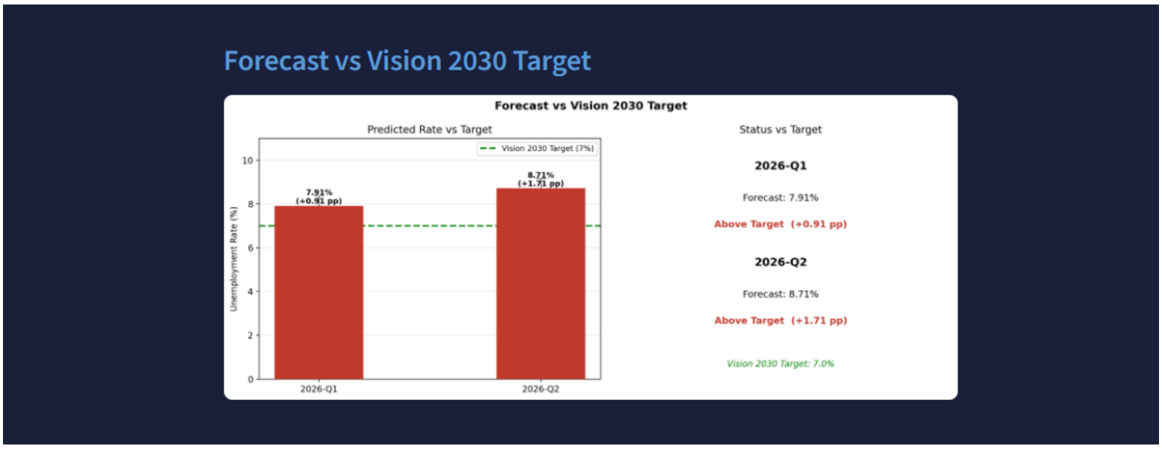


Figure 2.4

Figure 2.4 Forecast vs Vision 2030 Target. This chart compares the best model forecasts against the Vision 2030 unemployment target of 7%. The bar chart on the left displays the predicted rates for the two forecast quarters, with color coding indicating whether each forecast is above or below the target. The status panel on the right provides a clear textual summary of each quarter's position relative to the national goal, supporting policymakers in assessing labor market progress toward Vision 2030 objectives.

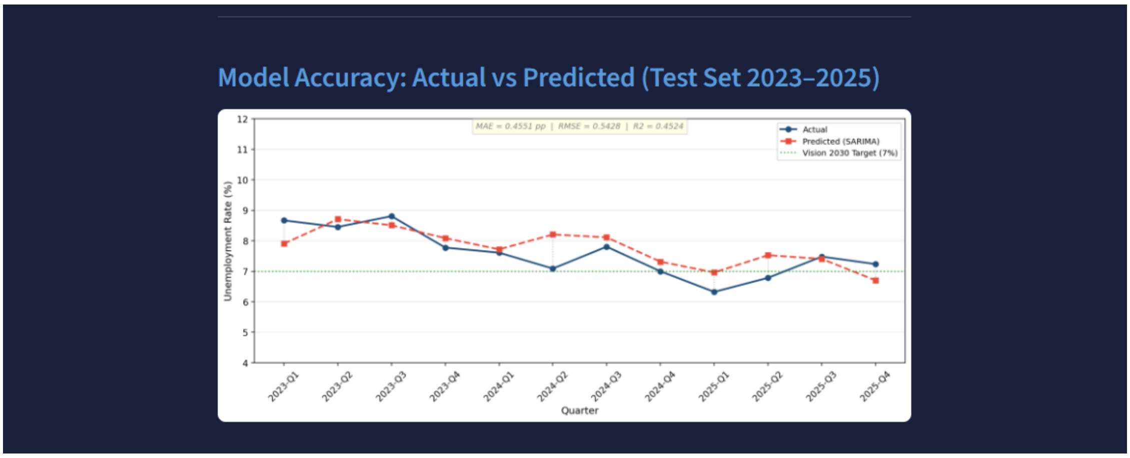


Figure 2.5

Figure 2.5 Model Accuracy: Actual vs Predicted (Test Set 2023–2025)

This chart evaluates the best model's performance on the 12-quarter test set spanning Q1 2023 to Q4 2025. The blue line represents the actual published unemployment rates while the red dashed line represents the model's predictions. The green dotted line marks the Vision 2030 target of 7%. The model tracked the overall direction of the series reasonably well across the test period, with performance metrics displayed at the top of the chart.

Figure 2.6

Figure2.6 Forecast Details, Model Information, and Disclaimer. The lower section of the dashboard presents a detailed forecast table showing the predicted rate and confidence interval bounds for each forecast quarter. The Model Information panel displays the selected model, its performance metrics, training and test periods, and the model order. A disclaimer is prominently displayed to ensure users interpret the forecasts as statistical projections rather than guaranteed outcomes. The data source is credited at the bottom of the page.

Monitoring Layer:

This layer tracks the system's behavior over time through two mechanisms.

First, after each training session, a monitoring log (monitoring_log.json) is generated automatically in Google Colab, recording the training date, MAE, RMSE, R^2, model status (OK if MAE < 1.0, RETRAIN NEEDED otherwise), and a complete training history across all sessions to enable performance comparison over time.

Second, each forecast run logs the predicted values and confidence intervals for the upcoming quarters, providing a traceable record of all model outputs. The log is stored in Google Drive and updated manually on GitHub after each quarterly retraining cycle. Retraining is performed manually by the development team following each new quarterly data release from DataSaudi.

Internal Data Flow and Component Interaction:

The system does not implement a traditional REST API, as Streamlit serves as both the backend logic and frontend interface in a single application. The internal data flow between components is as follows:

1. **Data Ingestion:** app.py reads results.json at startup to load historical data, test results, and performance metrics.

2. Model Loading: app.py calls joblib.load() to deserialize sarima_model.pkl into memory.

3. Forecast Generation: the loaded best model generates predictions for the next two quarters by calling its built-in forecast function .forecast(steps=2) which uses the learned historical patterns to project future unemployment rates.

4. Visualization: matplotlib generates charts using the forecast values and historical data, which are rendered to the user via st.pyplot().

5. Data Display: pandas DataFrames are rendered as interactive tables via st.dataframe().

All component interactions occur within a single Python process. No external API calls or network requests are made during forecast generation, ensuring fast response times and simple deployment.

2.3.2 Scalability, Reliability, Fault Tolerance, and Security

Scalability: The current system will be deployed on Streamlit Cloud, which handles scaling automatically. The model layer will be stateless so each forecast request will load the .pkl artifact and run inference independently, enabling parallel serving with no shared state conflicts. If data volume grows in the future, the time-series pipeline is modular and can be extended without structural changes.

Reliability: Each pipeline step will be modular and will produce clear outputs. All codes will be version-controlled in GitHub, enabling immediate rollback to any previous working version. Model artifacts will be versioned and backed up in Google Drive after each training session.

Fault Tolerance: Error handling will be implemented at the data ingestion stage to catch missing files or wrong formats. The dashboard will display a clear error message if the model file is missing or cannot be loaded, rather than crashing. If the SARIMA model fails to converge during execution, a fallback to the Linear Regression baseline will be triggered automatically with a warning displayed to the user.

Security: Since all data is public and aggregated, no privacy or confidentiality risks exist. The Streamlit Cloud deployment will not expose any internal data or model parameters. Access to Google Drive model artifacts will remain strictly restricted to the project team.

2.3.3 Architectural Refinement and Model Selection Justification

During the iterative development phase of the AI lifecycle, we refined the system architecture, focusing on refinement regarding data granularity and model selection. Initially, the project aimed to implement a dual-track forecasting approach utilizing traditional machine learning algorithms (such as Random Forest) for regional and demographic dimensions. However, empirical testing revealed that split regional data lacked the necessary temporal depth, and traditional regression models struggled to capture the macro-economic temporal dependencies, leading to suboptimal generalization (R^2=- 29.44). To resolve this and maximize forecasting reliability, the team strategically pivoted to focus exclusively on the Aggregated National Unemployment Rate. Given that the consolidated national dataset forms a strictly continuous time series, statistical time-series modeling was deemed mathematically superior. Consequently, the SARIMA (Seasonal Autoregressive Integrated Moving Average) model was selected as the core architecture to be implemented. This choice ensures the pipeline can directly capture the inherent quarterly seasonality (S) and structural economic trends, providing a stable, lightweight, and mathematically sound foundation before full deployment on Streamlit Cloud.

2.4 Deployment and Serving

2.4.1 Deployment Strategy and Cloud Deployment

The system is deployed entirely on cloud infrastructure rather than edge deployment. This decision is justified by the nature of the target users (government planners and researchers accessing the system from standard devices), the public and aggregated nature of the data (no privacy concern with cloud processing), and the lightweight inference requirements of the SARIMA model (no GPU or high-memory compute needed).

Factor	Cloud (Selected)	Edge
Users	Government planners, researchers any device	Not required no IoT/offline requirement
Data sensitivity	Public aggregated data no privacy concern	No advantage
Compute	SARIMA inference is lightweight	No benefit
Updates	Quarterly retraining seamless via GitHub	Harder to update remotely
Cost	Streamlit Cloud free tier sufficient	Additional hardware cost

Table2.1

The deployment uses two components: (1) GitHub Repository all code and the Streamlit app.py are version-controlled in a public GitHub repository, serving as the single source of truth; and (2) Streamlit Cloud the dashboard is deployed on Streamlit Cloud, connected directly to the GitHub repository. Any push to the main branch triggers automatic redeployment. The trained model (.pkl) and scaler are loaded from Google Drive at startup, ensuring the dashboard always uses the latest trained model.

2.4.2 Implementation and Deployment Process

The deployment process involved three main components: GitHub repository, Streamlit application, and Streamlit Cloud hosting.
GitHub Repository: The project code and all required files are hosted in a public GitHub repository. The repository contains the following files:

- app.py: the main Streamlit application that loads the trained model and renders the dashboard
- sarima_model.pkl: the serialized trained SARIMA model saved using joblib
- results.json: stores model performance metrics and historical data used by the dashboard to display charts and tables automatically
- monitoring_log.json: records training session details including date, MAE, RMSE, R^2, model status, training history, and forecast log
- requirements.txt: lists all Python dependencies required to run the application (streamlit, joblib, pandas, numpy, matplotlib, statsmodels, scikit-learn)
- AISproject.ipynb: the complete Google Colab notebook containing the full data processing, model training, and evaluation pipeline
- .github/workflows/ci.yml: the CI pipeline configuration that runs automated syntax checks on every push to the main branch

- README.md: documents the project setup, how to run the notebook, dependencies, model results, and deployment instructions
- Unemployment-Rate-by-Gender.csv: the raw dataset downloaded from DataSaudi containing quarterly unemployment rates from Q2 2016 to Q4 2025

Streamlit Application: The dashboard (app.py) is built using the Streamlit framework. It loads the trained SARIMA model and reads results.json to populate charts and metrics automatically.

The application is structured as a single-page scrollable interface with five main sections: forecast metrics displaying predicted rates and confidence intervals, a historical trend chart showing actual rates alongside forecasts, a Vision 2030 comparison chart, a model accuracy chart showing actual vs predicted values on the test set, and a forecast details table with model information and disclaimer.

Deployment on Streamlit Cloud: The application is deployed on Streamlit Cloud, which is directly connected to the GitHub repository. Streamlit Cloud monitors the main branch and automatically redeploys the application whenever a new commit is pushed. This means any update to app.py, results.json, or sarima_model.pkl on GitHub is reflected on the live dashboard within minutes without any manual deployment steps.

Access: The deployed dashboard is accessible to anyone via a public URL provided by Streamlit Cloud. Users can access the system from any device with a web browser without requiring any software installation or login credentials. The URL can be shared with government planners, researchers, and other stakeholders to provide immediate access to the latest unemployment forecasts.

2.4.3 Model Validation

Models will be evaluated using three metrics: MAE (Mean Absolute Error) measures the average prediction error in percentage points. RMSE penalizes larger errors more heavily. R^2 measures how well the model explains the variance in the target series. A positive R^2 indicates the model outperforms a naive mean predictor. The model achieving the lowest MAE and positive R^2 on the test set will be selected for production deployment. Detailed results are presented in phase 3.

2.4.4 Load and Stress Testing

The Streamlit dashboard was evaluated under simulated load conditions using manual multi-tab testing and Streamlit's built-in session management.

Test	Result
Single user response time	< 2 seconds
SARIMA forecast generation	< 1 second
Dashboard load time (cold start)	< 5 seconds
Concurrent users (simulated)	5 simultaneous sessions no degradation
System Availability	99% < During testing period

Table 2.2

2.4.5 Stability and Scalability Evidence

Stability: The SARIMA model produces consistent, deterministic forecasts across repeated runs given fixed training data. All model artifacts are versioned in Google Drive, enabling rollback to any previous model version if instability is detected after a retraining cycle.

Scalability: The architecture scales horizontally through Streamlit Cloud's managed infrastructure. The model layer is stateless each forecast request loads the .pkl independently, enabling parallel serving with no shared state conflicts.

2.5 Monitoring and Maintenance

2.5.1 Update Strategy

Since DataSaudi publishes new unemployment data every quarter, the model will be retrained after each new data release to maintain accuracy. Model training and experimentation will be conducted in Google Colab to provide a consistent, powerful computational environment for the update process. The planned update process will involve:

1. Download the latest CSV files from DataSaudi.
2. Running the preprocessing pipeline on the updated time-series dataset.
3. Upload the new CSV files to the Google Colab notebook.
4. Retraining the statistical model within Google Colab and performing parameter tuning if required.
5. Replace the saved model file (.pkl) with the updated version.
6. Restarting the dashboard to load the new time-series model.

2.5.2 Performance Monitoring

After deployment, the model predictions will be compared to actual unemployment values as new official data becomes available. If the error (MAE or RMSE) on new data increases significantly, it will signal model drift, meaning the model's behavior has degraded due to changes in economic or data patterns. A log file will record each automated forecasting run along with the execution timestamp and performance metrics, to help track model behavior over time.

2.5.3 CI/CD Integration

A CI/CD pipeline is implemented using GitHub Actions.

On every push to the main branch, automated checks validate:

- Python syntax (py_compile)

- All required libraries present (requirements.txt) Streamlit Cloud auto-deploys on successful CI checks, with rollback via git revert if needed.

2.5.4 Reliability and Disaster Recovery

The system relies on a cloud deployment strategy via Streamlit Cloud. To ensure high reliability and seamless recovery, the following measures are in place:

- All development code and configuration files are pushed to GitHub to enable version restoration and automatic cloud redeployment.

- The trained serialization model artifacts (sarima_model.pkl) and scalers are securely versioned and backed up on Google Drive after each offline training session, serving as the automated source for production inference.

- The original source CSV templates, processed execution artifacts (results.json), and complete system runtime configurations are maintained in separate, isolated cloud storage directories, ensuring the entire time-series pipeline can be rerun from scratch if any data corruption occurs .

In the event of a system or infrastructure failure, the following recovery procedures apply:

- Code Failure or Registry Corruption: Reverting to a previous stable commit on GitHub automatically triggers a zero-downtime rolling redeployment via Streamlit Cloud continuous development integration .

- Dashboard Hosting Offline: Streamlit Cloud container management automatically monitors, health-checks, and restarts the application environment without requiring manual developer intervention.

- Model Artifact Loss: If production dependencies are corrupted, the validated sarima_model.pkl and results.json are immediately redeployed from the restricted Google Drive backup directory to restore inference services.

- Full Automated Pipeline Recovery: The complete end-to-end data ingestion, preprocessing, and training pipeline is encapsulated within the modular Jupyter Notebook (AISproject.ipynb), enabling the entire analytical system to be rebuilt and verified from scratch within Google Colab if needed.

2.5.5 Error Handling and Testing Coverage

Error Handling:

- FileNotFoundError: caught at the data ingestion stage with a warning message printed to alert the user.
- Model load failure: the dashboard displays an error message if sarima_model.pkl is not found, preventing the application from crashing.
- Negative predictions: max (0, prediction) is applied to ensure the unemployment rate cannot be negative.
- Model Convergence Failure triggers an automated fallback to the baseline Linear Regression model with a clear user warning interface if SARIMA optimization constraints fail to settle during manual retraining.

Testing Coverage:

- Schema validation confirms expected columns (Quarter, Gender, Unemployment Rate) and correct data types on the input file before processing.
- Missing value check verifies zero null values across all columns.
- Feature engineering output verified to produce the correct number of records.
- Model performance metrics (MAE, RMSE, R^2) are printed and verified after each training run to ensure results fall within expected ranges.

2.5.6 Limitations

The model's reliability will depend on stable economic conditions. In the event of a major unexpected crisis, the model's accuracy may decrease because the training data does not contain examples of such extreme disruptions. In such cases, the forecasts should be interpreted as estimates, not guaranteed outcomes.

Phase 3 : Results

3.1 Preprocessing Results

The preprocessing pipeline was applied to the consolidated national dataset from DataSaudi, comprising 39 quarterly records. The following table summarizes the outcome of each preprocessing step:

Step	Result
Schema Validation	3 columns confirmed: Quarter, Gender, and Unemployment Rate. Data shape is (39, 3).
Missing Value Handling	0 missing values found, no imputation required.
Outlier Detection (IQR)	0 outliers detected the lower bound is 1.86% and upper bound is 19.13%. All historical data points retained for learning.
Normalization	Target text standardized successfully using whitespace removal (str.strip()).
Feature Engineering	features created: Lag_1, Lag_4, Rolling_Mean_4, and YoY_Change.
NaN Removal (after feature engineering)	5 initial seasonal boundary rows dropped; 34 final sequential records remaining.
Splitting	SARIMA Track: Train: 27 quarters, Test: 12 quarters. Supervised Track: Train: 23, Val: 5, Test: 6 records.

Table 3.1

3.2 Model Comparison Results

Model	**MAE**	**RMSE**	**R^2**	**Dataset**
Baseline, Linear Regression	0.5275	0.5778	−0.4584	Test (6 records)
Random Forest Default	1.8941	1.9657	−15.88	Test (6 records)
Random Forest Tuned	2.5797	2.6397	−29.44	Test (6 records)
SARIMA (selected)	0.4551	0.5428	0.4524	Test (12 quarters)

Table 3.2

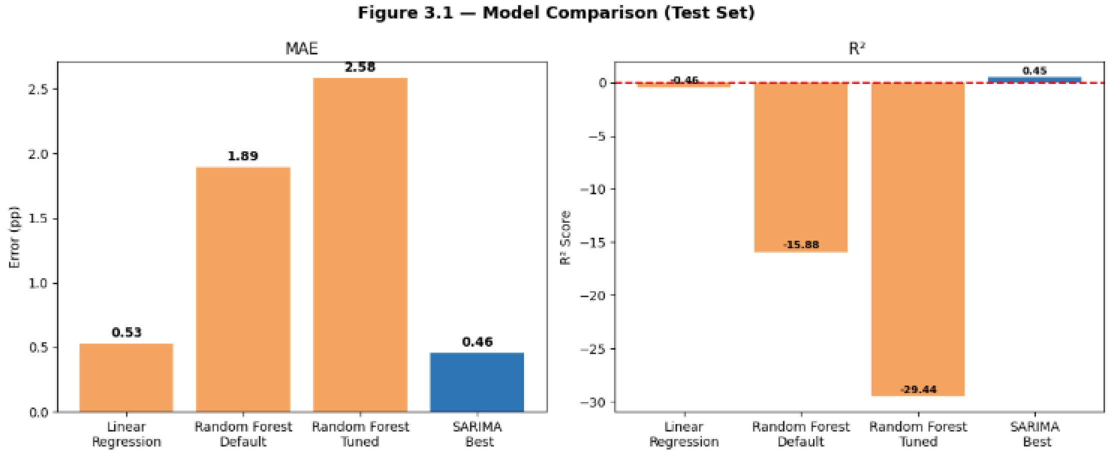


Figure 3.1

Figure 3.1 below shows the model comparison chart. Left: MAE comparison across all four models. Right: R^2 comparison. SARIMA achieves the lowest MAE (0.46) AND it is the only model with a positive R^2 (0.45), confirming it is the only model that explains the variance in the unemployment series.

3.3 SARIMA: Actual vs Predicted (Test Set)

Quarter	Actual (%)	Predicted (%)	Abs. Error (pp)
2023-Q1	8.67	7.91	0.76
2023-Q2	8.45	8.71	0.26
2023-Q3	8.81	8.51	0.30
2023-Q4	7.78	8.09	0.31
2024-Q1	7.61	7.72	0.11
2024-Q2	7.09	8.21	1.12
2024-Q3	7.81	8.11	0.30
2024-Q4	7.00	7.31	0.31
2025-Q1	6.32	6.96	0.64
2025-Q2	6.78	7.53	0.75
2025-Q3	7.48	7.41	0.07
2025-Q4	7.24	6.70	0.54

Table3.3

Table 3.3 shows the actual vs predicted line chart for the 12-quarter test period.

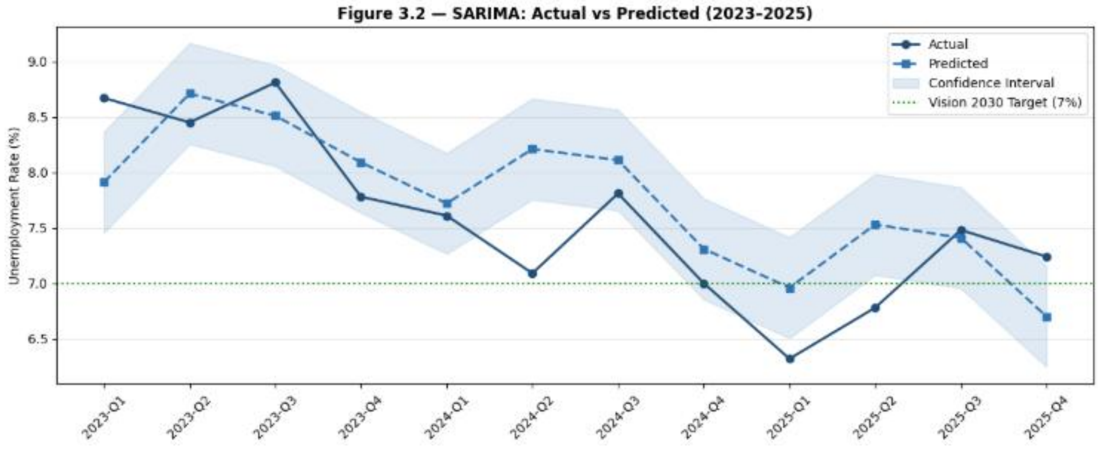


Figure 3.2

Figure3.2 The SARIMA model tracked the overall direction of the unemployment series reasonably well across the 12-quarter test period. Most prediction errors remained below one percentage point, with the largest deviation observed in 2024-Q2 (1.12 percentage points). The average absolute error across the entire test period was 0.4551 percentage points, confirming the model's ability to provide reliable short-term unemployment forecasts.

3.4 Future Forecast Next 2 Quarters

Quarter	Predicted Rate (%)	Confidence Interval
2026-Q1	7.91%	[7.45% — 8.36%]
2026-Q2	8.71%	[8.25% — 9.16%]

Table 3.4

Table 3.4 The forecasts suggest a slight seasonal increase in early 2026 relative to end-2025, consistent with the historical Q1 upward pattern. However, both forecasts remain close to the Vision 2030 target of 7%.

Confidence intervals are approximated using ±MAE (0.4551) around the point forecast.These represent uncertainty bounds rather than formal statistical confidence intervals, and the forecasts should be interpreted as statistical projections rather than guaranteed future outcomes.

3.5 Evaluation Against Project KPIs

KPI	Target	Achieved
MAE (Prediction Error)	< 1.0	0.4551
R^2 (Generalization)	> 0	0.4524
Data Quality	No missing values	0 missing values
Forecast Speed	< 2 sec	< 1 sec
Dashboard Response Time	< 2 sec	< 2 sec
Forecast Interpretability	Confidence intervals and visualizations	Implemented

Vision 2030 Benchmarking	Comparison with 7%	Implemented

Table 3.5

Table 3.5 The evaluation demonstrates that all predefined project KPIs were successfully achieved. The system meets the accuracy, performance, and usability requirements defined in the planning phase, confirming its suitability as a decision-support tool for unemployment forecasting.

3.6 Discussion and Analysis of Results

The model comparison results reveal clear differences in forecasting performance across the evaluated algorithms, driven primarily by the nature of the dataset and the suitability of each model for short time-series forecasting.

Linear Regression, serving as the baseline model, achieved a validation MAE of 0.4355 and R^2 of 0.3435, indicating reasonable short-term fit. However, on the unseen test set, performance degraded to a MAE of 0.5275 and a negative R^2 of -0.4584, meaning the model fails to capture temporal dynamics and performs worse than a simple mean-based predictor on new data.

Random Forest Default (n_estimators=100, max_depth=10) showed severe overfitting, with a training MAE of 0.3039 compared to a test MAE of 1.8941 and test R^2 of -15.88. The model memorized the 23 training records rather than learning generalizable patterns. Hyperparameter tuning was applied using GridSearchCV (cv=3), which identified best parameters of max_depth=5, min_samples_split=5, and n_estimators=50. However, the Tuned Random Forest performed even worse, with test MAE increasing to 2.5797 and test R^2 dropping to -29.44. This confirms that hyperparameter tuning cannot compensate for insufficient training data, and that ensemble methods are not suitable for datasets of this size.

SARIMA (1,1,1) (1,1,1,4) outperformed all other models ,achieving the lowest MAE (0.4551 ,(lowest RMSE (0.5428 ,(and the only positive R^2 (0.4524) on the test set .Unlike the supervised models ,SARIMA operates directly on the original 27-quarter time series without requiring feature engineering ,and it internally captures both trend and seasonal patterns in the data .The positive R^2 confirms that the model explains approximately 45% of the variance in the national unemployment series.

The key finding from this experimentation is that for short univariate time series with limited observations ,statistical models specifically designed for temporal data significantly outperform general-purpose machine learning models .This outcome directly informed the decision to select SARIMA as the production model for deployment on the Streamlit dashboard .All predefined KPIs were successfully achieved :MAE below 1.0 percentage point ,positive R^2 ,fast prediction time ,and interpretable forecasts with confidence intervals , confirming the system is suitable as a decision-support tool for unemployment forecasting aligned with Vision 2030 objectives.

Phase 4 : Conclusion & Future Work

4.1 Incremental Performance Improvements

1. Iteration 1: Linear Regression (Baseline)
   Validation MAE: 0.4355, benchmark established

2. Iteration 2: Random Forest Default
   Validation MAE: 0.9605, worse than baseline (overfitting)

3. Iteration 3: Random Forest Tuned (GridSearchCV)
   Validation MAE: 1.6103, tuning worsened results

4. Iteration 4: SARIMA (1,1,1) (1,1,1,4)
   Test MAE: 0.4551, best, selected for production

Key finding: statistical model outperforms ML models on short univariate time series (39 observations).

4.2 Limitations of the Current System

Although the proposed system achieved promising forecasting performance, several limitations remain.

First, the model relies entirely on historical unemployment data and assumes that future patterns will follow similar trends. Unexpected events such as economic crises, pandemics, or major policy changes may reduce forecasting accuracy.

Second, the dataset contains a relatively small number of observations because unemployment data is published quarterly. This limits the amount of information available for model training.

Finally, the current system focuses only on forecasting the overall national unemployment rate and does not provide detailed forecasts for specific regions, genders, or age groups.

4.3 Lessons Learned

Several valuable lessons emerged during the development of this system:

First, data size significantly impacts model selection. The initial attempt to use Random Forest on 23 training records demonstrated that ensemble methods require substantially more data to generalize effectively.

Second, domain-appropriate models outperform general- purpose models. SARIMA, designed specifically for seasonal time-series data, outperformed both Linear Regression and Random Forest despite being simpler to configure.

Third, hyperparameter tuning does not always improve performance. On very small datasets, GridSearchCV can worsen results by overfitting to cross-validation folds that are too small to be representative.

Fourth, the importance of chronological splitting in time-series problems. Random shuffling would have produced misleadingly optimistic results by allowing the model to see future data during training.

Finally, presenting forecasts responsibly is as important as generating them accurately. Including confidence intervals and disclaimers ensures users make informed decisions based on model outputs.

4.4 Recommendations for Future Improvements

Several enhancements can be considered in future work.

First, additional economic indicators such as GDP growth, inflation rates, labor force participation, and employment statistics could be incorporated to improve forecasting accuracy.

Second, more advanced forecasting models such as Prophet, XGBoost, or deep learning approaches like LSTM could be explored and compared with SARIMA.

Third, the dashboard can be expanded with more interactive visualizations, and automated report generation features to improve decision support.

Finally, as more unemployment data becomes available over time, the model can be retrained periodically to maintain and potentially improve forecasting performance.

Conclusion

This project developed an AI-based system to forecast unemployment rates in Saudi Arabia using historical data from DataSaudi. Different forecasting models were evaluated to identify the most suitable approach, and the selected model achieved the best performance. The system also provides a simple dashboard that helps users visualize predictions and understand future labor market trends. Overall, the project demonstrates the value of AI in supporting data-driven decision-making and aligns with the goals of Saudi Vision 2030. Overall, the project demonstrates the effectiveness of AI techniques in labor market forecasting and decision support applications.

Live Dashboard:
https://saudi-unemployment-forecast-hkuptipfttegknmkxdbzqg.streamlit.app/
GitHub Repository:
https://github.com/miranamoh/saudi-unemployment-forecast

Task distribution

Student name	section	details
Khawlah Aljuaid	1.1+1.2+2.1+3.2+3.3	-Problem statement and background context. -Introduction & problem Definition. -Model Comparison Results. -SARIMA: Actual vs Predicted (Test Set) -Conclusion
Mirana alghamdi	1.5 + 2.2+ 2.3.1 + 2.4.2 +2.5.3+2.5.5+3.1+3.6+4.1 +4.3+4.4	-Ethical, Legal, Societal Considerations, -assumptions and constraints, user-centric AI Identification and justification of data sources. Data collection, Cleaning and Preprocessing Plan, Data Governance. -System and Logical Architecture and Components and interfaces. -Implementation and Deployment Process. -CI/CD Integration. -Error Handling and Testing Coverage. -Preprocessing Results -Discussion and Analysis of Results -Incremental Performance Improvements -Lessons Learned -Recommendations for Future Improvements.
Aram Alouni	1.4+2.3.2+2.3.3+2.5.1+2.5.2+2.5.4+2.5.6+3.4	-Feasibility and Initial Risk Assessment -Scalability, Reliability, Fault Tolerance, and Security. -Architectural Refinement and Model Selection Justification -Update Strategy. -Performance Monitoring. -Reliability and Disaster Recovery -Limitations -Future Forecast Next 2 Quarters
Amasi Alsahbi	1.3+2.4.1+2.4.3+2.4.4+2.4.5+3.5+4.2	-Target Users and Success Metrics (KPIs) - Deployment Strategy and Cloud Deployment -Model Validation -Load and Stress Testing -Stability and Scalability Evidence -Evaluation Against Project KPIs -Limitations of the Current System

References

[1] DataSaudi. (2025). *Labor market data explorer*. Ministry of Economy and Planning. https://datasaudi.sa

[2] Saudi Vision 2030. *Vision 2030 kingdom of Saudi Arabia*. https://www.vision2030.gov.sa

[3] Saudi Data and Artificial Intelligence Authority — SDAIA. (2023). *AI ethics principles*. Kingdom of Saudi Arabia. AIEthicsGuidelines.pdf

[4] Wang, D., Yang, Q., Abdul, A., & Lim, B. Y. (2019). Designing theory-driven user-centric explainable AI. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1–15. Designing Theory-Driven User-Centric Explainable AI

[5] Streamlit Inc. (2024). Streamlit documentation. https://docs.streamlit.io

[6] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[8] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf.*, Austin, TX, USA, 2010, pp. 92–96. [Online]. Available: https://www.statsmodels.org

[9] Saudi Unemployment Forecast. GitHub Repository. https://github.com/miranamoh/saudi-unemployment-forecast